BLUE TEAM REVIEW

Review of the TestASP forum

Notes from a short defensive check

Date	30 October 2026	Analyst	Harshit
University	J.C. Bose University of Science and Technology, YMCA, Faridabad		
Course	B.Tech IT	Track	Blue Team
Site	testasp.vulnweb.com	Login data	Dummy values only
Target change	The original site named in the task was unreachable, so I used the alternate training site that was approved.		
Tools	Browser, Wireshark, Gobuster, SQLMap		

What stood out

The login form sent data in the clear. SQLMap confirmed an injectable forum id parameter, and the XSS test ran in the browser and opened an alert. Those are the findings I would follow up first.

I also tried three failed logins and ran a limited Gobuster scan. The two manually checked paths returned 404, while the scan found several responding paths. None of these checks demonstrated a directory listing. Three login attempts are too few to judge how the site would handle a sustained attack.

> **Where I would start:** require HTTPS throughout the site, fix the unsafe search output, then review the database query and login throttling. I used dummy credentials and did not retrieve table contents or customer records.

Scope

I kept the checks focused: three invalid logins, two manual path checks, one Gobuster directory scan, the SQLMap test of the forum id parameter, and one non-persistent XSS probe. I reviewed the browser traffic in Wireshark. The site identifies itself as a deliberately vulnerable demo forum; this is a short exercise, not a full audit.

AT A GLANCE

Key findings

Finding	Assessment result	Priority
Login transport	HTTP POST; dummy form fields are readable in the capture	High
Directory checks	Gobuster found responding files and directories; no directory listing was demonstrated	Review exposed paths
SQL injection	SQLMap confirmed injectable id parameter and listed six databases	High
Reflected input	Browser alert confirms script execution in the search result	High

Indicators from this exercise

These are lab traffic indicators for correlation, not confirmed attacker IoCs.

```
Host: testasp.vulnweb.com (44.238.29.244 in the saved capture)
Host paths: /Login.asp; /admin/; /backup/; /Search.asp; /showforum.asp?id=0
Gobuster hits include /vti_cnf/, /aspnet_client/, /avatars/, /cgi-bin/, /images/, /
jscripts/, /templates/, and common .asp pages
Capture frames: 22, 29, 36 (login); 4, 9 (path checks); 43, 47 (SQL-shaped probe / 500); 57,
64 (XSS probe / 200)
Dummy frame 22 form values: tfUName=blue_team_lab_user; tfUPass=invalid_test_only_1
SQLMap databases listed: acuforum, acuserservice, master, model, msdb, tempdb
SQLMap identified Microsoft SQL Server; 102 HTTP requests in the shown run
XSS test string: <svg onload=alert(1)> (URL encoded in the request)
```

MITRE ATT&CK context

T1595, Active Scanning, describes the Gobuster directory enumeration performed here. T1190, Exploit Public-Facing Application, is relevant to the SQL injection confirmed by SQLMap and the reflected XSS test. T1110.1, Password Guessing, is relevant to the three invalid login attempts, though those few test submissions are not evidence of a real brute-force attack. These mappings describe the assessment actions and detection context, not an outside actor or attribution.

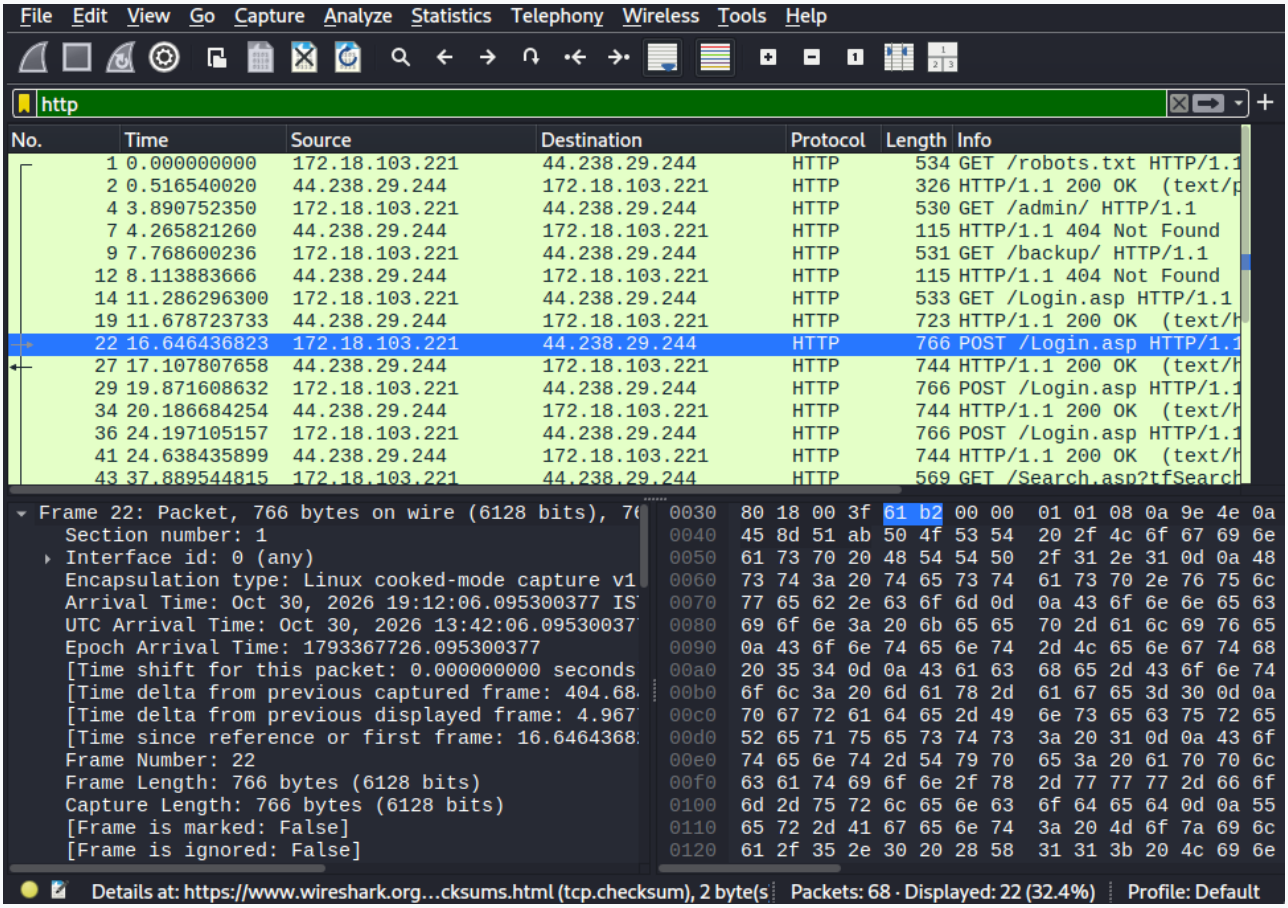
Scope note: The screenshots document the assessment activity. They do not show access to customer records; SQLMap was used to enumerate database names only.

LOGIN CHECKS

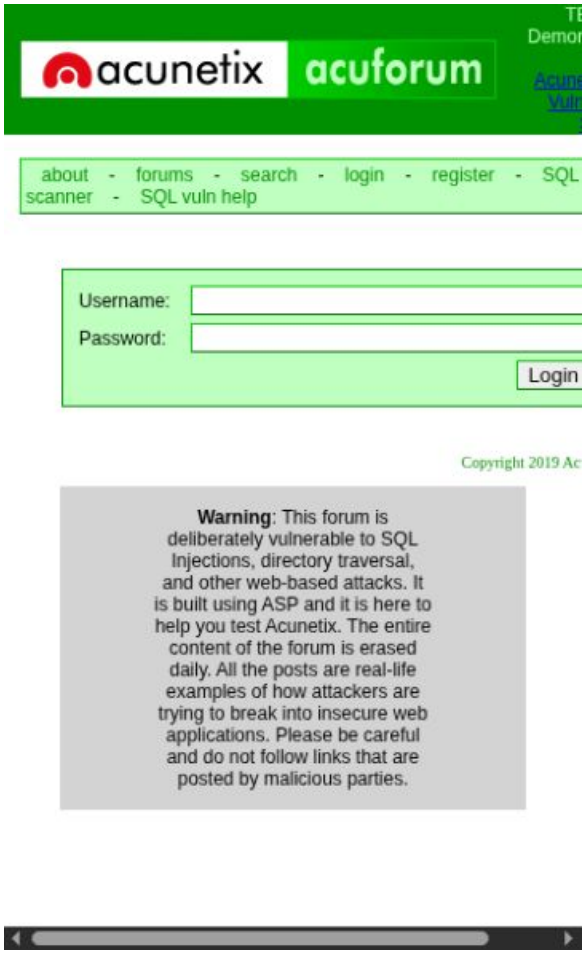
The login is sent without encryption

Anyone on the network could read a login sent this way

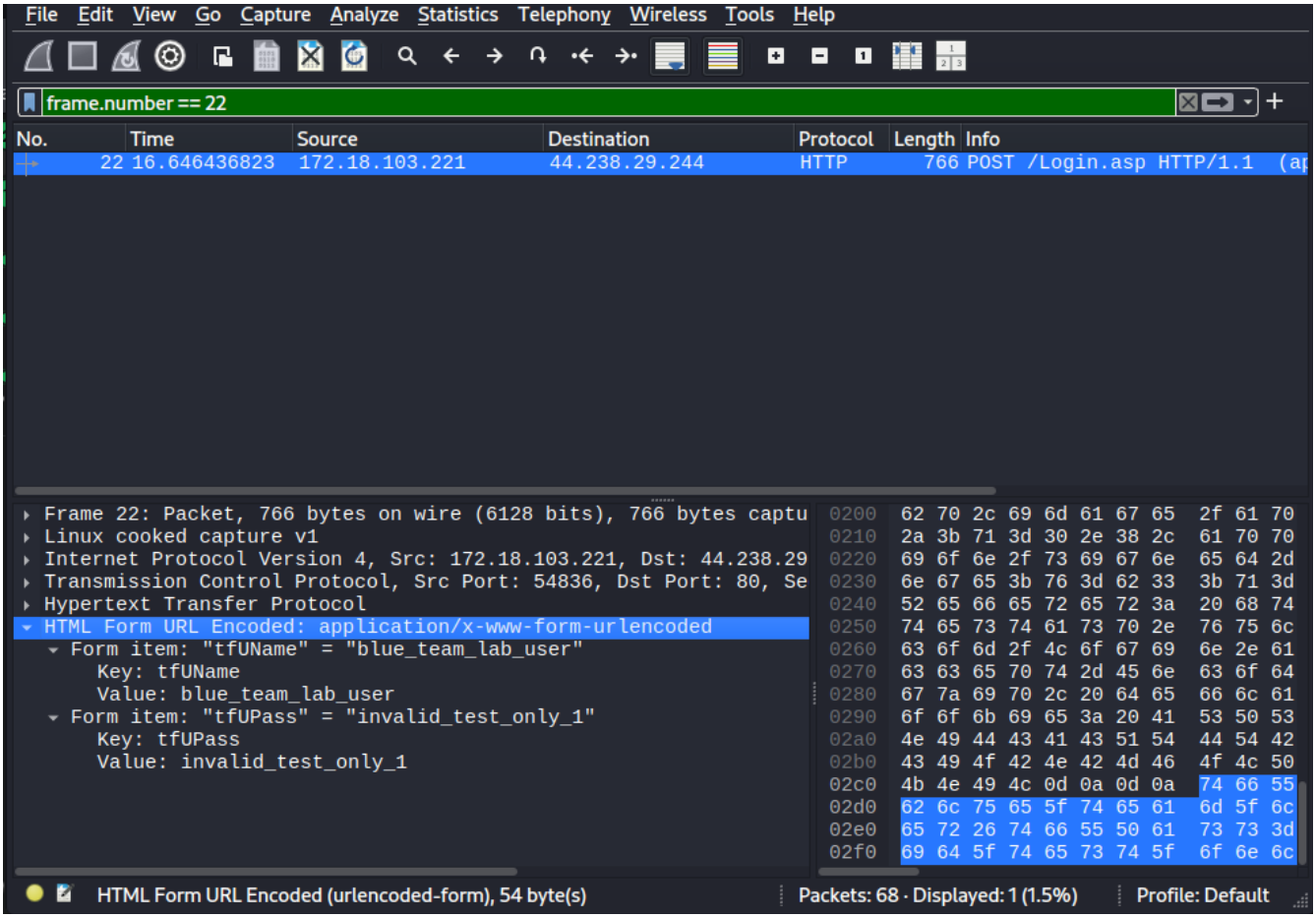
I submitted dummy credentials to the login form. Frame 22 is a form-encoded POST over plain HTTP on port 80. The expanded Wireshark view shows the form fields in clear text. The values are dummy test credentials. Anyone able to observe traffic sent this way could read them.



Wireshark with the HTTP display filter applied and frame 22 selected. The packet list identifies frame 22 as the POST to /Login.asp. The expanded form fields appear in the next screenshot.



Browser view of the login page over HTTP, showing the address bar without a secure connection indicator.



Wireshark frame 22 with “HTML Form URL Encoded” expanded. It shows tfUName=blue_team_lab_user and tfUPass=invalid_test_only_1; both values are dummy credentials.

The fix is to serve the site only over HTTPS and redirect HTTP requests. Enable HSTS after HTTPS is working throughout the site. Do not use real credentials on this training target.

Three failed logins, with no visible slowdown

I tried three invalid logins. The site replied “Invalid login” and accepted the next attempts without a visible delay, CAPTCHA, or lockout. That is worth checking, though three attempts do not tell us how it would behave under a sustained attack.

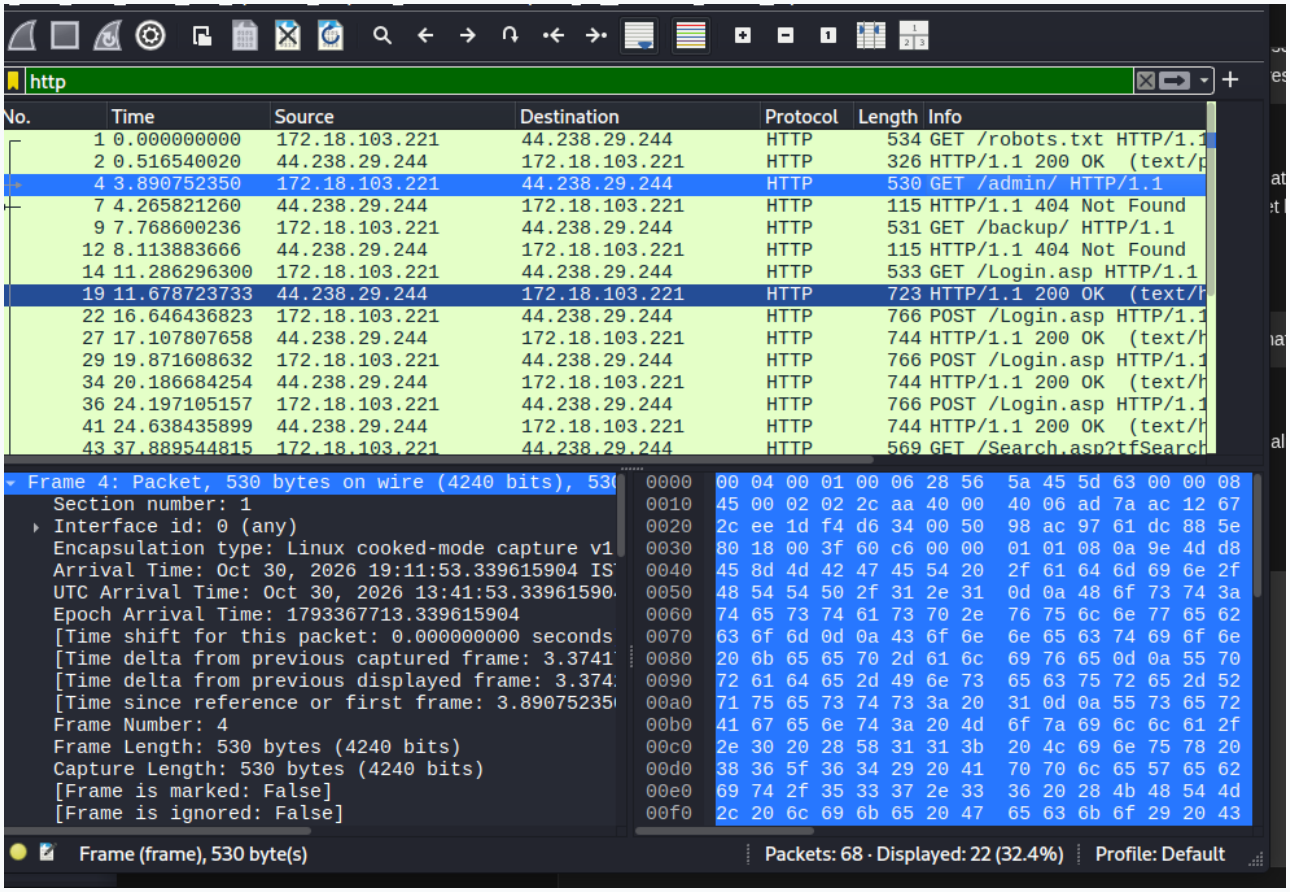
I would alert on repeated failures from the same source or against the same account, then confirm the site rate-limits or locks out repeated attempts.

FINDINGS

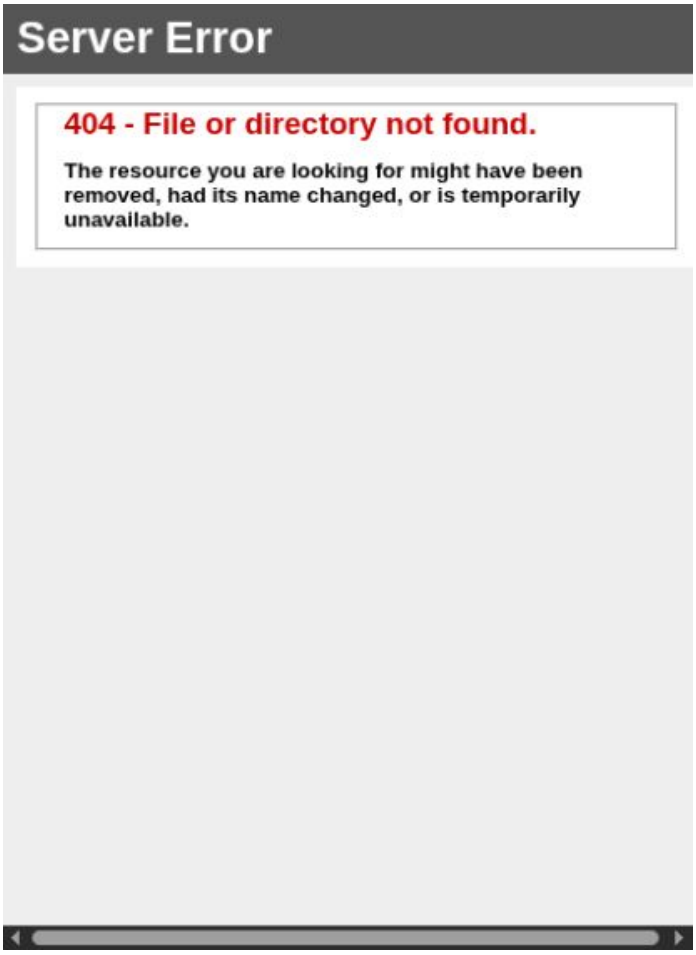
A couple of common paths

No directory listing was demonstrated

I first checked `/admin/` and `/backup/`; both returned 404, and `/robots.txt` did not list disallowed paths. I then ran Gobuster with the common directory wordlist, five threads, and ASP/TXT/BAK extensions. It found several responding paths, including a number of 301 directory redirects, 200 responses for common ASP pages, and a 403 for `/cgi-bin/`. That is useful surface inventory, but it does not show that a directory listing or sensitive file was exposed.



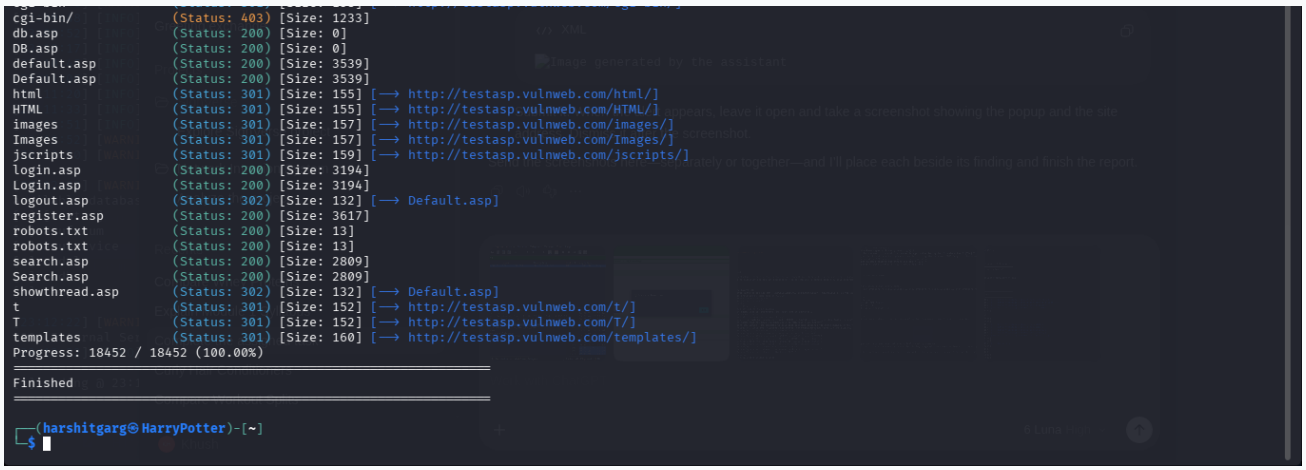
Wireshark filtered to HTTP with the /admin/ request selected. The packet list also shows its 404 response and the second tested path returning 404.



Browser response when opening the checked /admin/ path.

```
┌──(harshitgarg㉿HarryPotter)-[~]
└─$ gobuster dir -u http://testasp.vulnweb.com/ \
  -w /usr/share/wordlists/dirb/common.txt \
  -t 5 -x asp,txt,bak
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://testasp.vulnweb.com/
[+] Method:                  GET
[+] Threads:                 5
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              asp,txt,bak
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
_vti_cnf             (Status: 301) [Size: 159] [--> http://testasp.vulnweb.com/_vti_cnf/]
aspnet_client        (Status: 301) [Size: 164] [--> http://testasp.vulnweb.com/aspnet_client/]
avatars              (Status: 301) [Size: 158] [--> http://testasp.vulnweb.com/avatars/]
cgi-bin              (Status: 301) [Size: 158] [--> http://testasp.vulnweb.com/cgi-bin/]
cgi-bin/             (Status: 403) [Size: 1233]
db.asp               (Status: 200) [Size: 0]
DB.asp               (Status: 200) [Size: 0]
default.asp          (Status: 200) [Size: 3539]
Default.asp          (Status: 200) [Size: 3539]
html                 (Status: 301) [Size: 155] [--> http://testasp.vulnweb.com/html/]
HTML                 (Status: 301) [Size: 155] [--> http://testasp.vulnweb.com/HTML/]
images               (Status: 301) [Size: 157] [--> http://testasp.vulnweb.com/images/]
Images               (Status: 301) [Size: 157] [--> http://testasp.vulnweb.com/Images/]
jscripts             (Status: 301) [Size: 159] [--> http://testasp.vulnweb.com/jscripts/]
login.asp            (Status: 200) [Size: 3194]
Login.asp            (Status: 200) [Size: 3194]
logout.asp           (Status: 302) [Size: 132] [--> Default.asp]
register.asp         (Status: 200) [Size: 3617]
robots.txt           (Status: 200) [Size: 13]
robots.txt           (Status: 200) [Size: 13]
search.asp           (Status: 200) [Size: 2809]
Search.asp           (Status: 200) [Size: 2809]
```

Gobuster command and results in progress, using the common wordlist and the limited extensions shown.



Completed Gobuster run (18,452 of 18,452 entries). The results show discovered paths and their HTTP statuses; a response alone does not confirm sensitive exposure.

What I would check next: review the listed endpoints for unintended files, confirm directory indexes are disabled, and remove or restrict anything that should not be public.

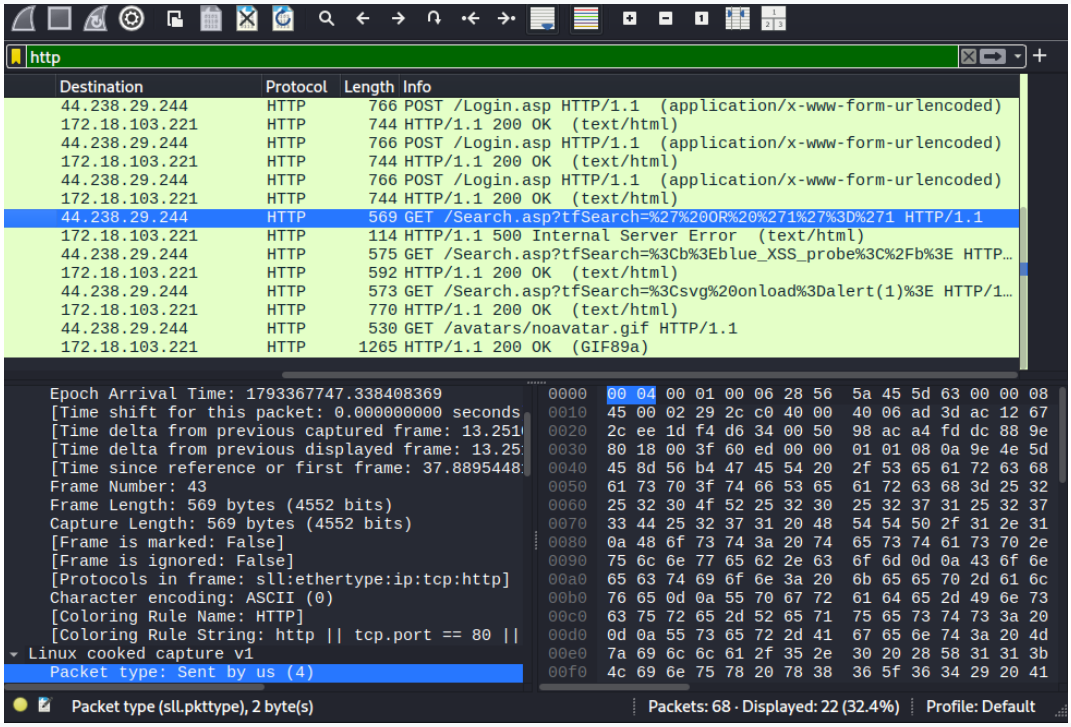
SEARCH CHECK

A search probe returned a server error

CONFIRMED SQL INJECTION ON THE FORUM ID PARAMETER

The earlier search probe returned a generic HTTP 500, which on its own was inconclusive. I then ran SQLMap against `showforum.asp?id=0`. It reported the GET parameter `id` as vulnerable to boolean-based blind and time-based blind SQL injection, identified Microsoft SQL Server, and listed six database names: `acuforum`, `acuserservice`, `master`, `model`, `msdb`, and `tempdb`. I did not dump table contents or records.

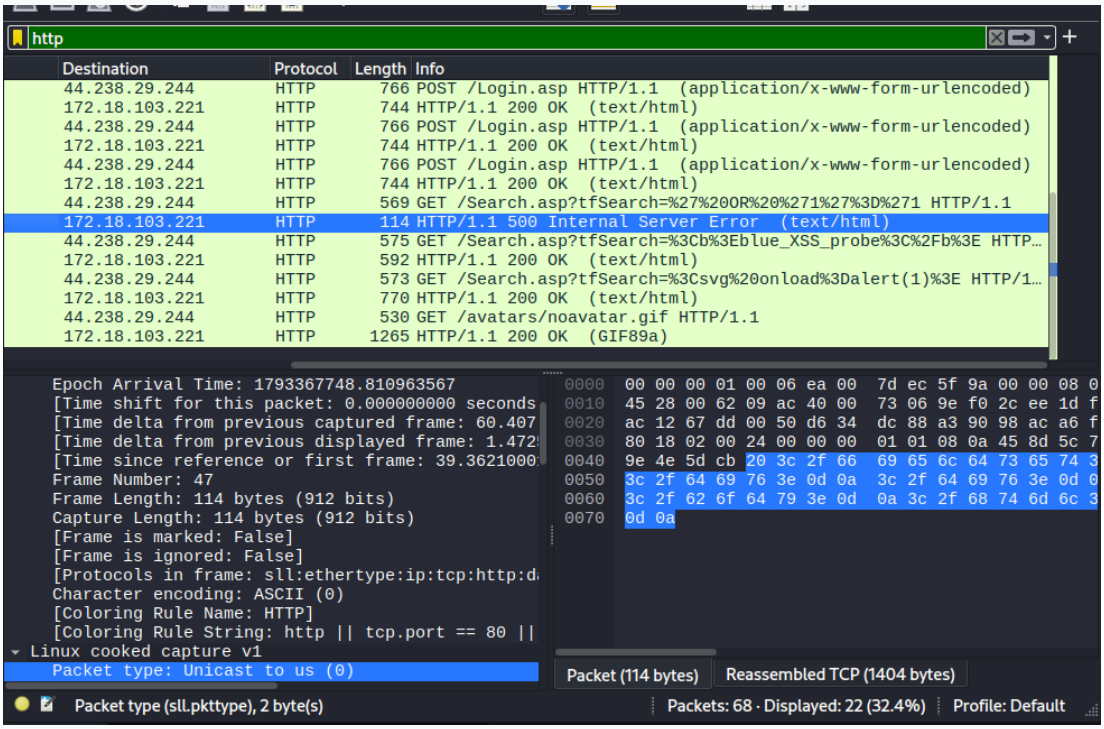
Follow-up: parameterize the `id` query, review the database account permissions, and check application/database logs for unexpected queries.



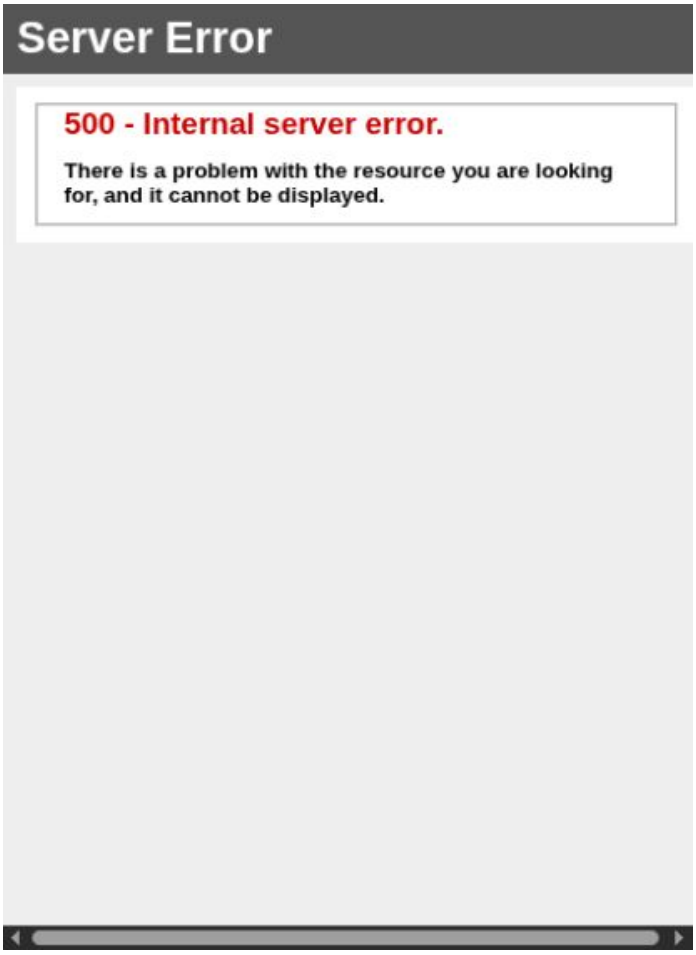
Wireshark with frame 43 selected: the search request containing the encoded test input.

The server returned an error

Frame 47 is the reply to the request on the previous page. It is a generic HTTP 500 response.



Wireshark with frame 47 selected: the HTTP 500 response to the search probe.



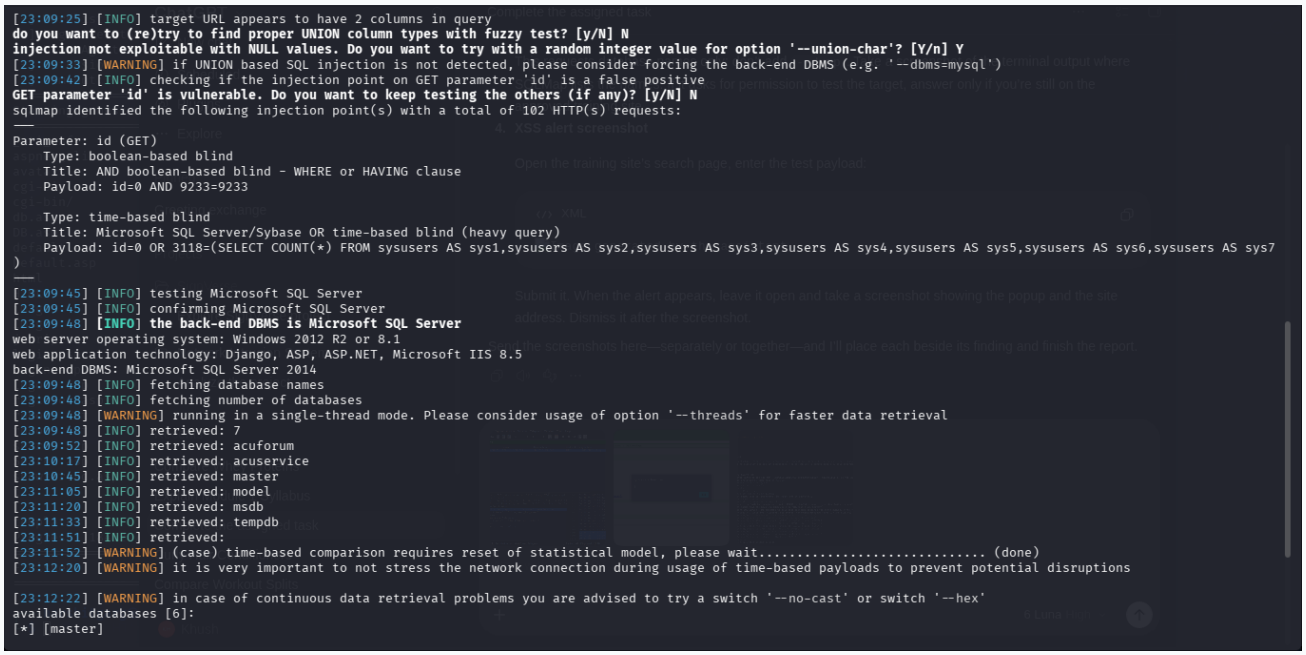
Browser response after the search probe. This error was only an indicator; SQLMap provided the later confirmation on a separate endpoint.

The id parameter was injectable

The terminal screenshots show the SQLMap test and the resulting database-name list. No table rows or customer records were requested.



SQLMap testing `showforum.asp?id=0`. The run identifies the Microsoft SQL Server backend and reports blind injection on the id parameter.



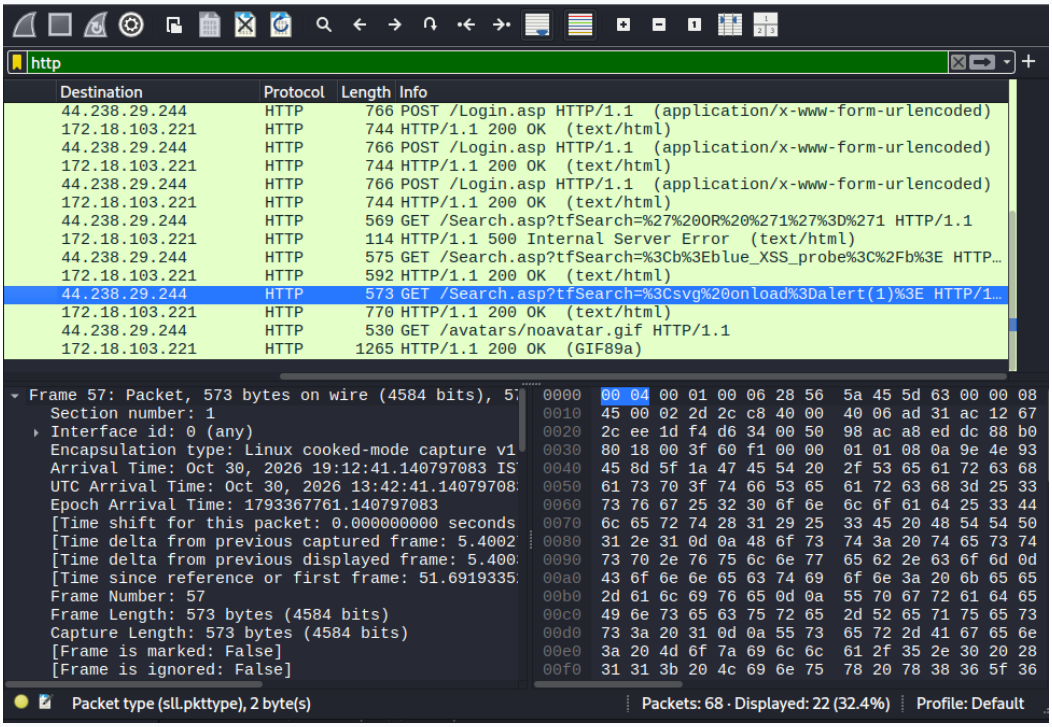
SQLMap database enumeration output. The visible list contains six names: acuforum, acuservice, master, model, msdb, and tempdb.

REFLECTED XSS

The search text ran in the browser

I put a simple SVG event-handler test into the search field. The request is frame 57 and the reply is frame 64. The browser opened an alert showing “1”, which confirmed the script ran. I dismissed it; nothing was saved to the forum. The new screenshot captures the alert while it was open.

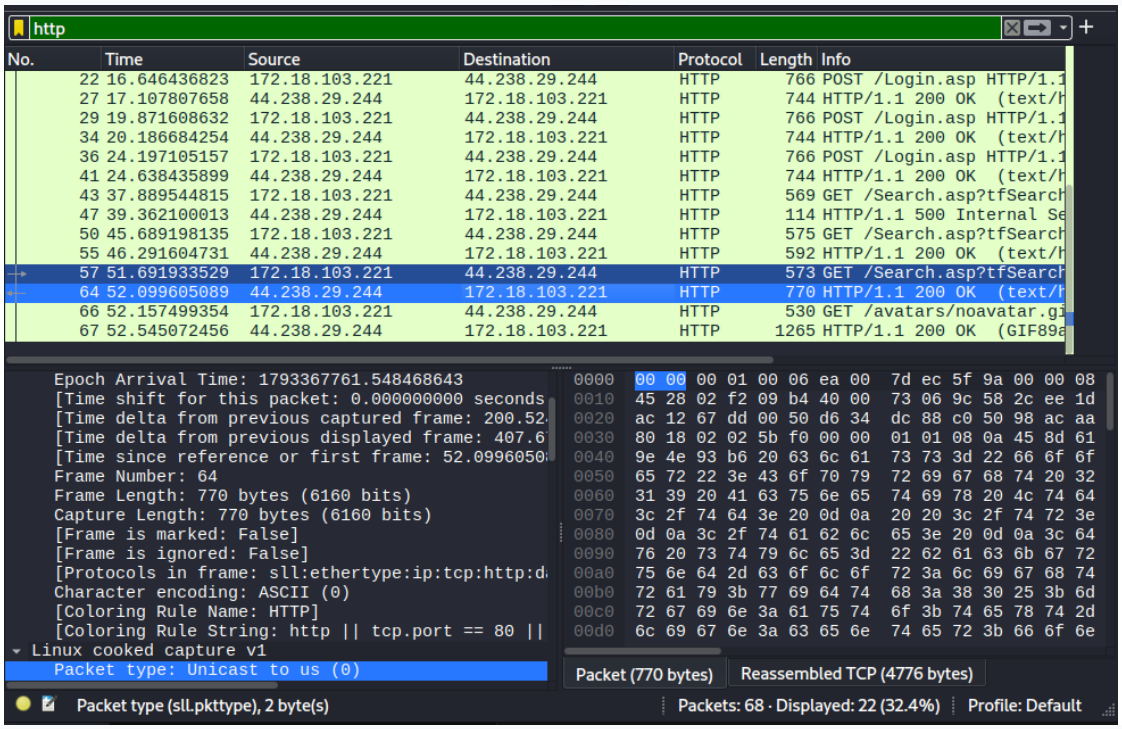
A crafted link could make a visitor’s browser run script on this site. The page should escape search text before displaying it. A content security policy is useful as a backup, but it does not replace that fix.



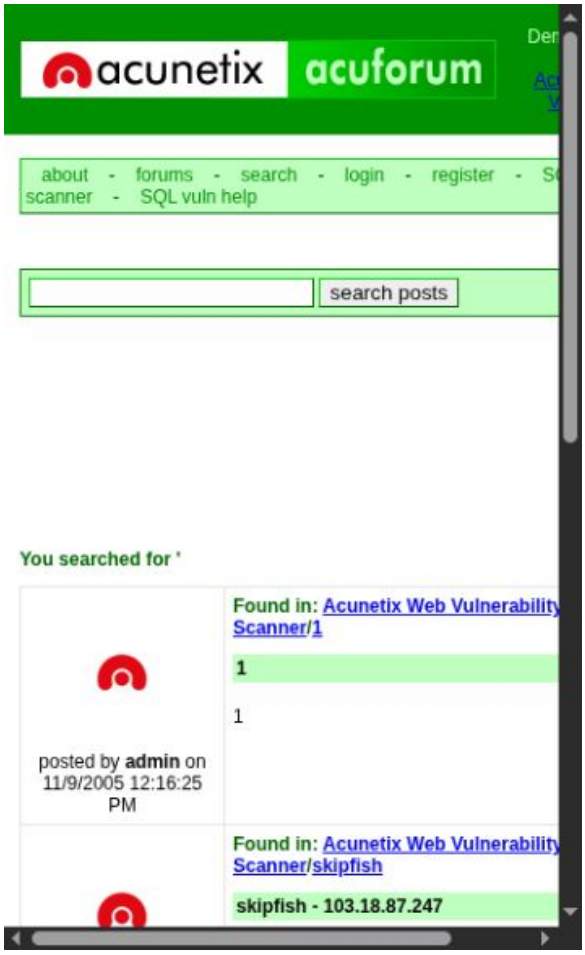
Wireshark with frame 57 selected: the search request contains the encoded SVG event-handler test.

The reply to the search request

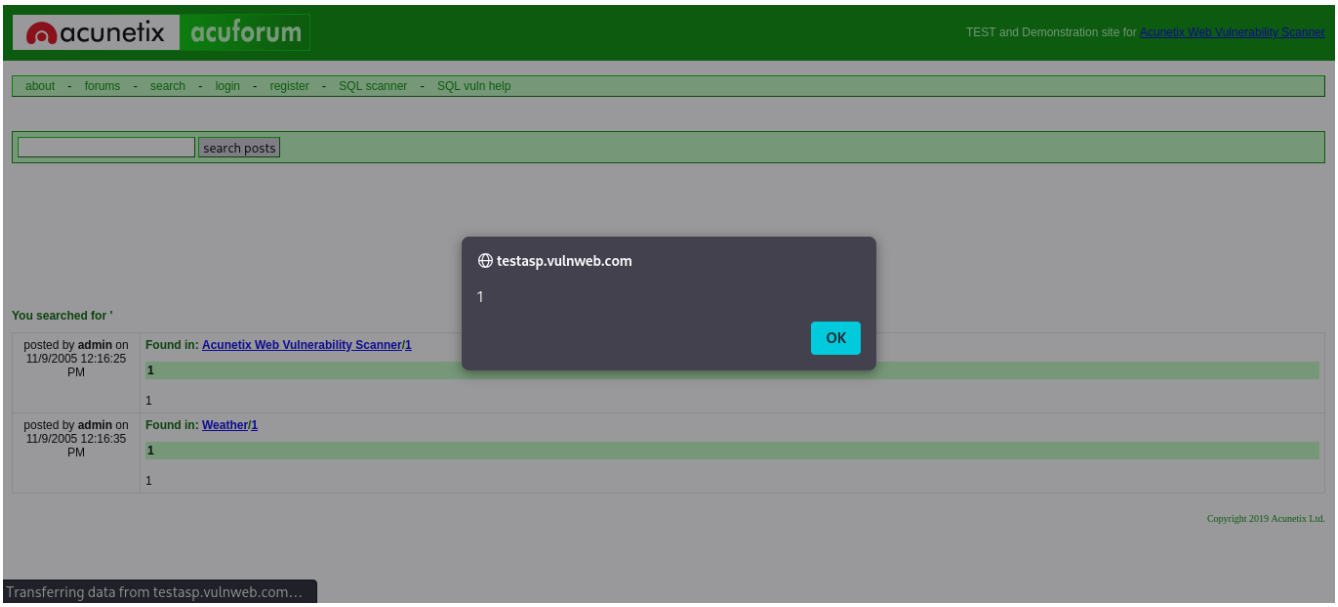
Frame 64 is the HTTP 200 response to the request shown on the previous page.



Wireshark with frame 64 selected: the response to the XSS test request.



Search result after the alert was dismissed, showing the reflected content.



Retaken browser screenshot with the alert showing “1” still open on testasp.vulnweb.com.

REVIEW NOTES

What I would fix first

1. **Require HTTPS.** Check that the login form no longer sends fields over plain HTTP.
2. **Fix the injectable queries.** Parameterize the `showforum.asp` id lookup and search queries, encode reflected search text, then retest.
3. **Check login protections.** Verify rate limiting and alert on repeated failed attempts.
4. **Watch for path probing.** Look for quick bursts of requests to many missing paths, especially backup or configuration files.

About the evidence

The attached capture has 68 packets for the browser-based login, manual path checks, and search probes. The Gobuster and SQLMap activity was captured separately in the terminal screenshots, so those requests are not part of this pcap. I did not have server logs or a live SIEM feed, so the monitoring ideas below are suggestions based on this traffic, not alerts that actually fired.

```
Relevant frames in testasp_training_capture.pcapng
22, 29, 36   POST /Login.asp   (three dummy login attempts)
7, 12        responses to /admin/ and /backup/   (404)
43, 47       SQL probe and HTTP 500 response
57, 64       XSS probe and HTTP 200 response
```

One note on the packet views

The expanded Wireshark screenshot shows frame 22 and both form fields. The username and password are dummy test values only.

Guidance consulted: OWASP Web Security Testing Guide (credential transport and reflected XSS); OWASP SQL Injection Prevention Cheat Sheet.